THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD

COURSE TITLE	Software Construction & Development
COURSE CODE	SE-3101
LAB TITLE	Activity Diagram
STUDENT NAME	MAHDI ALI NAHVI
STUDENT ROLL NUMBER	2024-SE-16
INSTRUCTOR NAME	Engr. Dr. Asma Javaid
SUBMISSION DATE	October 27, 2026
LAB NUMBER	02

Objective

The objective of this lab is to understand and practise UML behavioural modelling. Behavioural diagrams describe **how a system acts over time** rather than how it is structured. In this lab two kinds of behavioural diagrams are used:

- **Activity diagram with swimlanes** – to model a workflow in which several actors take part (Q1: the foodPanda online food-ordering process).
- **State (state machine) diagram** – to model the life cycle of one object as it reacts to events (Q2: a toaster, Q3: an elevator/lift).

The diagrams were drawn with Python (matplotlib) and Graphviz, and are included as images in this report. The editable source scripts are supplied with the lab files (see README).

Summary of Tasks

No.	Task	Diagram Type
Q1	Activity diagram of the foodPanda application (online food order app) using swimlanes	Activity diagram with swimlanes
Q2	State diagram for a Toaster; list activities within each state; identify triggered and triggerless events/actions	State diagram
Q3	State machine diagram of an Elevator/Lift	State machine diagram

Question 1: foodPanda Activity Diagram with Swimlanes

1.1 Problem Statement

Draw an activity diagram by visualizing the process of the foodPanda application (online food order app) using swimlane.

1.2 Introduction

An **activity diagram** is a UML flow chart that shows the sequence of actions in a process, the decisions that change the path, and where the process begins and ends. When several people or systems take part in the same process, **swimlanes** (also called partitions) are added. Each swimlane is a vertical or horizontal band that belongs to one actor, so the diagram shows not only *what* happens but also *who* is responsible for each action.

Ordering food through foodPanda involves four parties, so the diagram is divided into four swimlanes:

Swimlane	Responsibility in the process
Customer	Opens the app, logs in, chooses restaurant and items, pays, receives the food and rates the order.
foodPanda App (System)	Validates the order, processes the payment, forwards the order to the restaurant, handles cancellation/refund and assigns a rider.
Restaurant	Receives the order, decides to accept or reject it, prepares the food and marks it ready.
Delivery Rider	Accepts the delivery request, picks up the food and delivers it to the customer.

1.3 Notation Used

Symbol	Meaning
Filled black circle	**Initial node** – where the process starts.
Rounded rectangle	**Action** – one step of the process performed by the owner of the lane.
Diamond	**Decision node** – the flow splits depending on a yes/no condition (e.g. payment successful?).
Arrow	**Control flow** – the order in which actions are executed; arrows crossing a lane boundary show a hand-over between actors.
Circle with inner dot	**Final node** – the process ends.
Coloured vertical band	**Swimlane** – groups the actions of one actor.

1.4 Activity Diagram

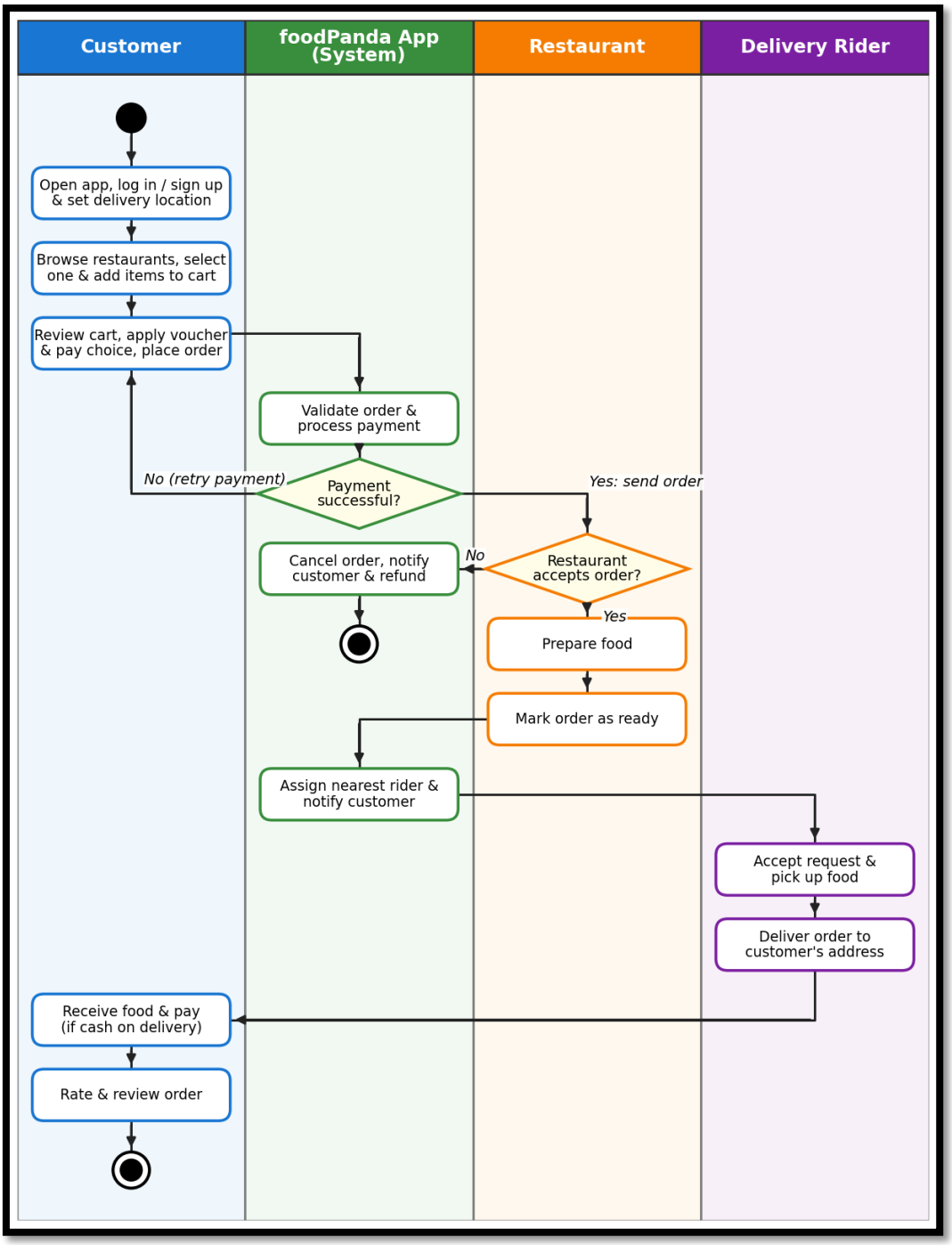


Figure 1: Activity diagram with swimlanes for the foodPanda online food-ordering process

1.5 Explanation of the Diagram

The process is explained below in the order in which it occurs, with the swimlane responsible for each step shown in brackets.

1. **Start and ordering (Customer).** The process starts when the customer opens the app, logs in or signs up and sets a delivery location. The customer then browses the restaurants, selects one, and adds items to the cart. Finally the customer reviews the cart, applies a voucher if available, chooses a payment method and places the order.

2. **Payment (foodPanda App).** The order passes to the system lane, which validates it and processes the payment. A decision node *Payment successful?* follows.
3. **Payment failure (alternative flow).** If the answer is **No**, the flow returns to the customer, who can correct the details or choose a different payment method and place the order again. This forms a loop in the diagram.
4. **Restaurant decision (Restaurant).** If the payment is successful (**Yes**), the system sends the order to the restaurant. The restaurant decides at the node *Restaurant accepts order?*
5. **Rejection (alternative flow).** If the restaurant cannot accept the order (**No**), the system cancels the order, notifies the customer and refunds the payment. The process ends at the final node in the System lane.
6. **Preparation (Restaurant).** If the order is accepted (**Yes**), the restaurant prepares the food and marks the order as ready.
7. **Rider assignment (foodPanda App).** Once the order is ready, the system assigns the nearest available rider and notifies the customer that the order is on its way.
8. **Delivery (Delivery Rider).** The rider accepts the request, picks up the food from the restaurant and delivers it to the customer's address.
9. **Completion (Customer).** The customer receives the food and pays the rider if cash on delivery was selected. The customer may then rate and review the order, and the process reaches the final node.

Key points. The diagram has two decision nodes, giving the main (successful) flow and two alternative flows (payment failure and restaurant rejection). Every arrow that crosses a swimlane boundary represents a hand-over of work from one actor to another, which is the main advantage of using swimlanes.

Question 2: State Diagram of a Toaster

2.1 Problem Statement

Draw a State diagram for a Toaster. List activities within each state. Identify triggered and triggerless events/actions.

2.2 Introduction

A **state diagram** (UML state machine) models the different conditions (states) an object can be in and how it moves from one state to another. Its main parts are:

- **State** – a condition during which the object behaves in a particular way (e.g. *Toasting*).
- **Transition** – a change from one state to another, written as **event [guard] / action**.
- **Event** – something that happens and causes a transition (e.g. the user pushes the lever).
- **Guard** – a condition in square brackets that must be true for the transition to fire.
- **Activities inside a state** – **entry /** (runs once when the state is entered), **do /** (runs continuously while in the state) and **exit /** (runs once when the state is left).

Transitions are of two kinds. A **triggered** transition fires only when an external event arrives (a button press, a sensor signal). A **triggerless** transition (also called a *completion transition*) has no event; it fires automatically when the activity of the source state has finished (for example when a timer expires), or it is the initial transition from the start node.

2.3 States and Their Activities

The toaster was analysed as having six states. The activities performed inside each state are listed below.

State	entry / (on entering)	do / (while in state)	exit / (on leaving)
Idle (Off)	Heating elements OFF; indicator light OFF	Wait for bread to be inserted	–
Bread Loaded (Ready)	Show current browning level	Monitor browning dial and lever	–
Toasting	Lock lever down; heating elements ON; start timer	Heat the bread; count down the timer	Heating elements OFF
Ejecting	Release lever spring	Push toast upwards	–
Toast Ready	Sound the ‘ding’	Wait for the toast to be removed	–
Safety Cut-off	Cut power; warning light ON	Cool down	–

2.4 State Diagram

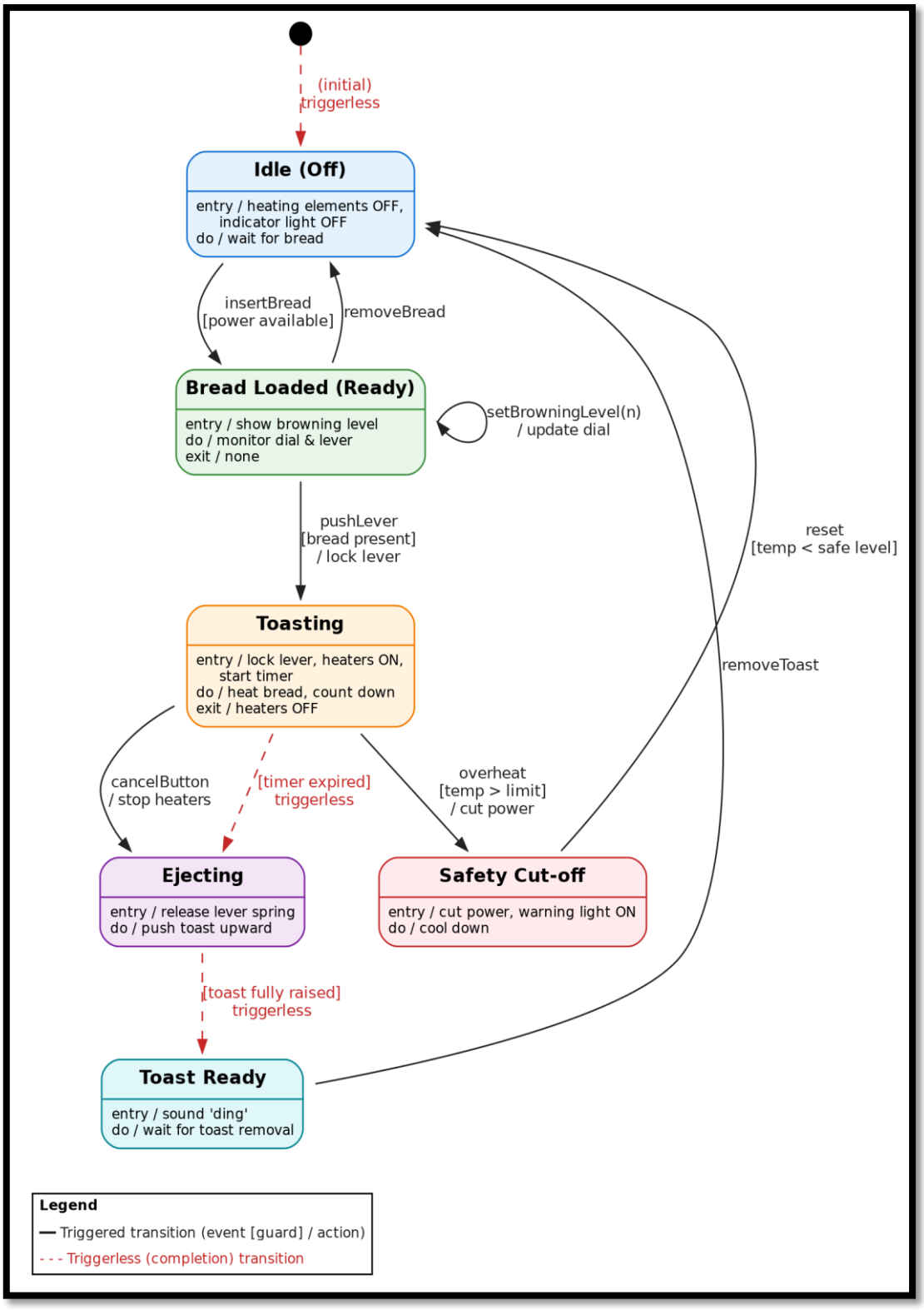


Figure 2: State diagram of a toaster (solid arrows = triggered, dashed red arrows = triggerless)

2.5 Transitions: Triggered and Triggerless

The table below lists every transition of the diagram and classifies it.

#	From		To	Event (trigger)	Guard / Action	Type
1	Initial node	→	Idle	– (none)	Toaster powered	**Triggerless**
2	Idle	→	Bread Loaded	insertBread	[power available]	Triggered
3	Bread Loaded	→	Idle	removeBread	–	Triggered
4	Bread Loaded	↺	Bread Loaded	setBrowningLevel(n)	/ update dial	Triggered
5	Bread Loaded	→	Toasting	pushLever	[bread present] / lock lever	Triggered
6	Toasting	→	Ejecting	cancelButton	/ stop heaters	Triggered
7	Toasting	→	Ejecting	– (none)	[timer expired]	**Triggerless**
8	Toasting	→	Safety Cut-off	overheat	[temp > limit] / cut power	Triggered
9	Ejecting	→	Toast Ready	– (none)	[toast fully raised]	**Triggerless**
10	Toast Ready	→	Idle	removeToast	–	Triggered
11	Safety Cut-off	→	Idle	reset	[temp < safe level]	Triggered

2.6 Triggered vs. Triggerless Events/Actions

Triggered events/actions (need an external event):

- insertBread, removeBread, setBrowningLevel(n), pushLever, cancelButton, overheat, removeToast and reset. The actions written after the slash on these transitions (lock lever, stop heaters, cut power, update dial) run **only because** the triggering event occurred.

Triggerless events/actions (happen automatically):

- **Initial transition** into Idle when the toaster is powered.
- **Toasting → Ejecting** when the timer expires. The *do* activity of Toasting finishes, so the toast pops up without any user input.
- **Ejecting → Toast Ready** once the toast is fully raised.
- The **entry, do and exit activities** inside every state (e.g. heaters ON on entering Toasting, heaters OFF on leaving) are also executed automatically by the state itself and not by a separate event.

2.7 Explanation of the Diagram

The toaster starts in **Idle (Off)** with the heating elements off. When the user inserts bread and power is available, it moves to **Bread Loaded (Ready)**, where the user may change the browning level any number of times (an internal transition that does not leave the state) or remove the bread to return to Idle. Pushing the lever down with bread present moves it to **Toasting**: the lever is locked, the heating elements are switched on and the timer starts.

From Toasting there are three ways out. The normal path is **triggerless**: when the timer expires the toaster moves to **Ejecting** by itself. The user can also press the cancel button, which is a **triggered** transition to the same state. If the temperature exceeds the safe limit, the **Safety Cut-off** state cuts the power; after cooling down and a reset the toaster returns to Idle. In Ejecting, the lever spring is

released and the toast pops up, after which the toaster reaches **Toast Ready**, sounds a ding and waits until the toast is removed, returning to Idle.

Question 3: State Machine Diagram of an Elevator/Lift

3.1 Problem Statement

Draw State Machine diagram of Elevator/Lift.

3.2 Introduction and Assumptions

An elevator is a reactive system: it continuously reacts to events such as passengers pressing floor buttons, floor sensors detecting the car, door sensors, a load sensor and an emergency button. A state machine diagram is therefore a natural way to describe its behaviour. The following assumptions were made to keep the model clear:

- A single elevator car serves several floors, and the car **never moves while the doors are open or opening**.
- Floor requests (from inside the car or from the hall) are queued; the guard conditions *request above* and *request below* refer to this queue.
- The doors stay open for a dwell time of 5 seconds unless the user presses the open or close button.

3.3 Identified States

State	Description	Activities (entry / do / exit)
Idle	Car is stopped at a floor, doors closed, no pending request.	entry / motor OFF, brake ON; do / wait for requests
Moving Up	Car is travelling to a higher floor.	entry / brake OFF, motor UP; do / update floor display; exit / motor OFF, brake ON
Moving Down	Car is travelling to a lower floor.	entry / brake OFF, motor DOWN; do / update floor display; exit / motor OFF, brake ON
Doors Opening	Car has stopped at the target floor and the doors are opening.	entry / start door motor (open); do / watch door sensor
Doors Open	Doors are fully open so passengers can enter or leave.	entry / start 5 s dwell timer; do / let passengers in/out; exit / stop timer
Doors Closing	Doors are closing.	entry / start door motor (close); do / watch obstruction sensor
Overload	Passenger load exceeds the capacity; the car must not move.	entry / buzzer ON, show ‘Overload’; do / hold doors open; exit / buzzer OFF
Emergency Stop	Emergency button pressed or power failure while moving.	entry / apply brake, alarm ON, call help; do / hold car stationary

3.4 State Machine Diagram

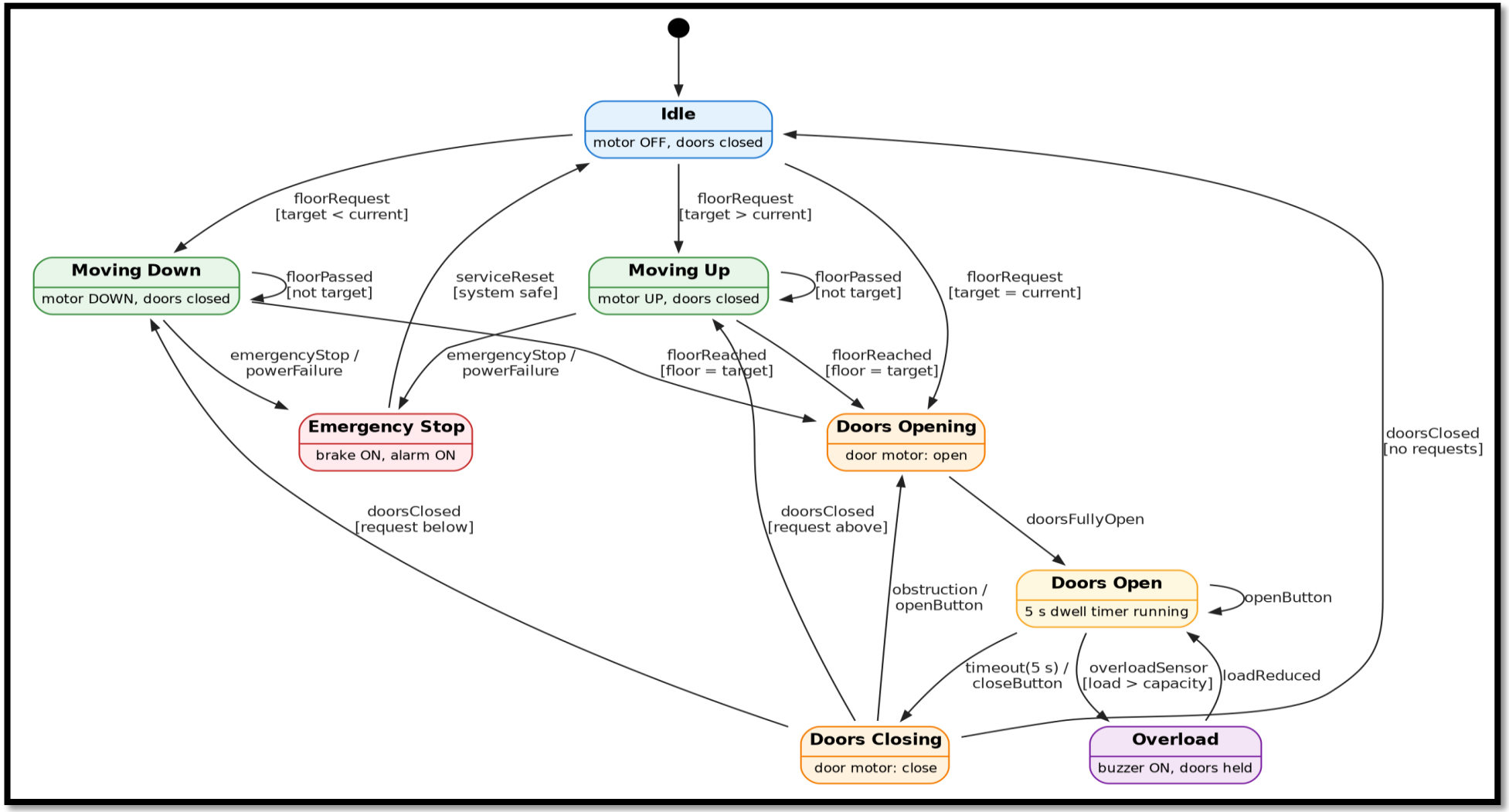


Figure 3: State machine diagram of an elevator/lift

3.5 Transitions

From → To	Event	Guard	Action	Notes
Initial → Idle	–	–	–	Triggerless start
Idle → Moving Up	floorRequest	target > current	start motor up	Request for higher floor
Idle → Moving Down	floorRequest	target < current	start motor down	Request for lower floor
Idle → Doors Opening	floorRequest	target = current	start door motor	Already at that floor
Moving Up ↺ Moving Up	floorPassed	floor ≠ target	update display	Passing an intermediate floor
Moving Down ↺ Moving Down	floorPassed	floor ≠ target	update display	Passing an intermediate floor
Moving Up/Down → Doors Opening	floorReached	floor = target	stop motor, apply brake	Arrival at the target floor
Doors Opening → Doors Open	doorsFullyOpen	–	start dwell timer	Sensor signal
Doors Open ↺ Doors Open	openButton	–	restart timer	Passenger holds the doors
Doors Open → Doors Closing	timeout(5 s) or closeButton	–	–	Normal closing
Doors Open → Overload	overloadSensor	load > capacity	buzzer ON	Safety check
Overload → Doors Open	loadReduced	load ≤ capacity	buzzer OFF	Passenger leaves
Doors Closing → Doors Opening	obstruction or openButton	–	reverse door motor	Safety re-open
Doors Closing → Idle	doorsClosed	no pending requests	–	Nothing left to serve
Doors Closing → Moving Up	doorsClosed	request above	–	Continue upwards
Doors Closing → Moving Down	doorsClosed	request below	–	Continue downwards
Moving Up/Down → Emergency Stop	emergencyStop or powerFailure	–	apply brake, alarm	Safety
Emergency Stop → Idle	serviceReset	system safe	alarm OFF	Technician resets

3.6 Explanation of the Diagram

After power-up the elevator enters **Idle**, where the motor is off, the brake is on and the doors are closed. A floor request moves it in one of three directions depending on the guard: **Moving Up** if the requested floor is above, **Moving Down** if it is below, or directly to **Doors Opening** if the car is already at that floor.

While in a moving state the elevator stays in the same state each time a floor is passed (self-transition) and only updates the floor display. When the floor sensor reports the target floor, the motor stops, the brake is applied and the elevator goes to **Doors Opening**, then **Doors Open** when the door sensor confirms the doors are fully open. The 5-second dwell timer runs in this state. A passenger pressing *open* restarts the timer; if the load sensor reports more than the allowed capacity, the car enters **Overload**, sounds a buzzer and keeps the doors open until the load is reduced.

When the timer expires, or the close button is pressed, the car enters **Doors Closing**. If an obstruction is detected (or the open button is pressed) the doors reverse back to Doors Opening, which is an important safety feature. Once the doors are closed, the next state depends on the request queue: Idle if there are no pending requests, otherwise Moving Up or Moving Down. At any time while moving, an emergency stop or power failure takes the car to **Emergency Stop**, where the brake is applied and the alarm sounds, until a technician performs a service reset and the elevator returns to Idle.

3.7 Example Scenario

A passenger on floor 1 presses the button for floor 3:

1. Idle → **Moving Up** (floorRequest, target 3 > current 1).
2. Floor 2 is passed: self-transition on Moving Up (display shows 2).
3. Floor 3 is reached: Moving Up → **Doors Opening** (floorReached, floor = target).
4. Doors Opening → **Doors Open**; the passenger exits within 5 seconds.
5. Timeout: Doors Open → **Doors Closing**; doors close with no obstruction.
6. doorsClosed with no pending requests: Doors Closing → **Idle**.

Conclusion

In this lab, three real-world processes were modelled with UML behavioural diagrams. The foodPanda ordering process was shown as an **activity diagram with swimlanes**, which clearly separates the work of the customer, the app, the restaurant and the rider. The toaster was modelled as a **state diagram** with entry, do and exit activities and a clear distinction between **triggered** and **triggerless** transitions. The elevator was modelled as a **state machine** with guards, self-transitions and safety states. The lab showed that activity diagrams suit workflows involving several actors, while state diagrams suit objects whose behaviour depends on their current state.